CHAPTER 8

ARRAYS

CHAPTER ESSENCE AT A GLANCE:
Master contiguous homogeneous indexed memory structures enabling systematic sequential and multi-dimensional storage, search algorithms, and bubble and selection sorting routines.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Contiguous Memory Allocation:** Arrays group homogeneous elements together into consecutive memory locations indexed by zero-based integer offsets.

• **Index-Based Direct Addressing:** Any element in a one-dimensional array is accessed in constant time O(1) by computing base address plus index multiplied by element byte size.

• **Linear versus Binary Search Complexity:** Linear search examines unsorted elements sequentially in O(n) time, whereas binary search halves sorted search intervals in logarithmic time O(log n).

• **Two-Dimensional Matrix Layout:** Two-dimensional arrays map mathematical matrices into linear memory sequentially row-by-row under row-major storage ordering.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Address of } A[i] = \text{Base} + i \times \text{sizeof(Type)}$$

1D Array Element Address Calculation

$$\text{Address of } A[i][j] = \text{Base} + (i \times C + j) \times \text{sizeof(Type)}$$

Row-Major 2D Matrix Memory Address

$$\text{Linear Search: } O(n) \quad \longleftrightarrow \quad \text{Binary Search: } O(\log_2 n)$$

Search Algorithm Time Complexity Comparison

$$\text{Bubble Sort Comparisons} = \frac{n(n-1)}{2} \to O(n^2)$$

Worst-Case Sorting Comparison Metric

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Array index out of bounds: in an array declared as int a[10];, valid indices range from 0 to 9. Accessing a[10] causes undefined memory corruption.

• **Alert:** Binary search strictly requires the array elements to be pre-sorted in ascending or descending order before searching.

• **Alert:** In 2D array declarations such as int matrix[][3] = {...};, the column dimension is mandatory and cannot be omitted.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. What is an array in C++? State two key advantages of using arrays. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Definition:** A collection of homogeneous data elements of the same data type stored in contiguous memory locations under a common identifier name. **[1 Mark]**

• **Advantages:** (1) Enables storing large volumes of related data under a single variable name; (2) Provides fast random access to any element in O(1) constant time using its index. **[1 Mark]**

2. Explain how a one-dimensional array is declared and initialized in C++ with an example. [Score: 2 Marks]

Answer & Key Valuation Steps:

• **Syntax:** dataType arrayName[size]; specifies element type and number of items. **[1 Mark]**

• **Initialization Example:** int scores[5] = {85, 92, 78, 90, 88}; stores five integers at indices scores[0] through scores[4]. **[1 Mark]**

3. Compare Linear Search and Binary Search on precondition, algorithm logic, and time complexity. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Precondition:** Linear search works on both sorted and unsorted arrays; Binary search strictly requires the array to be sorted. **[1 Mark]**

• **Search Mechanism:** Linear search checks elements sequentially from first to last; Binary search repeatedly compares the search key with the middle element, halving the search space. **[1 Mark]**

• **Time Complexity:** Linear search has worst-case time O(n); Binary search has logarithmic time O(log2 n). **[1 Mark]**

4. Explain the Bubble Sort algorithm to arrange an array of N integers in ascending order. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Algorithmic Concept:** In each pass, compare adjacent pairs of elements (a[j] and a[j+1]); if the left element is greater than the right, swap them. **[1 Mark]**

• **Pass Invariant:** After pass 1, the largest element bubbles up to the highest index. After pass i, the i largest elements are in their final sorted positions. **[1 Mark]**

• **Total Passes & Comparisons:** Requires n - 1 passes with total comparisons n(n - 1) divided by 2, yielding $O(n^2)$ time complexity. **[1 Mark]**

5. Explain Row-Major Order memory storage for a two-dimensional array matrix[3][4] of integer data type. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Row-Major Concept:** In C++, 2D array elements are arranged in linear physical memory sequentially row-by-row: all elements of row 0 are stored, followed by row 1, and then row 2. **[1 Mark]**

• **Address Formula:** Address of element matrix[i][j] = Base Address + (i * ColumnCount + j) * sizeof(DataType). **[1 Mark]**

• **Application:** For matrix[3][4] with Base Address 2000 and 4 bytes per int, matrix[2][1] address = 2000 + (2 * 4 + 1) * 4 = 2000 + 36 = 2036. **[1 Mark]**

6. Write a complete C++ program to read N integers into an array and find the largest and smallest elements. [Score: 4 Marks]

Answer & Key Valuation Steps:

• **Input & Traversal:** int a[100], n; cin >> n; for(int i=0; i<n; i++) cin >> a[i]; **[1 Mark]**

• **Finding Min & Max:** int minVal = a[0], maxVal = a[0]; for(int i=1; i<n; i++) { if(a[i] > maxVal) maxVal = a[i]; if(a[i] < minVal) minVal = a[i]; } **[2 Marks]**

• **Output & Boxed Result:** cout << 'Max=' << maxVal << ' Min=' << minVal; Boxed Result: Single linear pass O(N) evaluation. **[1 Mark]**

7. Write a C++ program to compute the transpose of a 3x3 square matrix. [Score: 3 Marks]

Answer & Key Valuation Steps:

• **Input Matrix:** int mat[3][3], trans[3][3]; for(int i=0; i<3; i++) for(int j=0; j<3; j++) cin >> mat[i][j]; **[1 Mark]**

• **Transpose Logic:** Interchanging rows and columns: for(int i=0; i<3; i++) for(int j=0; j<3; j++) trans[j][i] = mat[i][j]; **[1 Mark]**

• **Display & Boxed Result:** Displaying trans matrix formatted row-by-row. Boxed Result: trans[j][i] = mat[i][j] across all coordinates. **[1 Mark]**